

Universidad Nacional de San Martín

TP INTEGRADOR:

Materia: **Bases de datos**

Alumnos: Chavez Leandro - Avila Lautaro - Giuggiolini Enzo Manuel - Millauro Nahuel - Mastrangelo Thiago

Docentes: Rodrigo Casco , Carlos Gabriel Flores

Fecha: 13/11/2026

ÍNDICE:

1. Descripción del problema elegido.....3

2. Descripción y origen del dataset.....3

3. Análisis inicial de los datos.....4

4. Proceso de normalización.....6

5. Diagrama Entidad-Relación..... 9

6. Modelo Relacional.....10

7. Diseño de la solución NoSQL.....13

8. Justificación de la utilización de NoSQL..... 14

9. Arquitectura general de la aplicación.....15

10. Explicación del backend polimórfico..... 17

11. Tecnologías utilizadas..... 19

12. Instrucciones para ejecutar el proyecto.....21

13. Ejemplos de funcionamiento.....22

14. Comparación entre ambas implementaciones..... 26

15. Problemas encontrados y decisiones tomadas.....28

16. Conclusiones.....29

1. Descripción del problema elegido.

Desde el primer momento teníamos la idea de hacer un juego para que sea un proyecto entretenido y nos motive a trabajar más allá de cumplir con una lista de requisitos. Luego de debatir varias opciones, inspirándonos en el formato de los juegos diarios en línea como Wordle o Críptico, decidimos desarrollar un GeoGuessr centrado en lugares y espacios culturales de la Ciudad Autónoma de Buenos Aires.

La propuesta consiste en mostrarle al jugador una vista de calle en 360° para que explore el entorno y marque en un minimapa en qué lugar de la ciudad cree que se encuentra.

La elección de las bases de datos fue bastante directa para el equipo. Por un lado, una base relacional resulta indispensable para estructurar la información y llevar el control de usuarios. Por otro lado, un modelo no relacional se alinea con la arquitectura habitual de este tipo de plataformas, permitiendo manejar de forma eficiente las coordenadas GeoJSON de los lugares y calcular las distancias en el mapa.

2. Descripción y origen del dataset.

Una vez definida la idea del proyecto, necesitábamos un dataset representativo con lugares culturales de la Ciudad de Buenos Aires. Para esto, recurrimos al portal oficial de datos abiertos del Gobierno de la Ciudad proveído por la cátedra y buscamos registros sobre puntos turísticos e infraestructura cultural. No fue difícil dar con una opción que se adecue a nuestras necesidades, seleccionando finalmente un conjunto de datos con más de 3.000 registros disponibles para trabajar.

El dataset original contaba con una gran variedad de atributos organizados en 30 columnas distintas:

- **fid:** Identificador único del lugar dentro del archivo original.
- **ESTABLECIMIENTO:** Nombre oficial del punto de interés o espacio cultural.
- **FUNCIÓN PRINCIPAL:** Tipo principal de lugar (ej. Museo, Teatro, Bar, Centro Cultural).

- **SUBCATEGORÍA:** Detalle o clasificación específica del establecimiento.
- **BARRIO:** Barrio porteño donde se encuentra ubicado el espacio.
- **COMUNA:** Número de comuna de CABA correspondiente a la ubicación.
- **CALLE / ALTURA / DIRECCIÓN:** Ubicación catastral y domicilio físico.
- **LATITUD / LONGITUD:** Coordenadas geográficas para posicionar el lugar en el mapa.
- **TELÉFONO / MAIL / WEB / REDES:** Información de contacto y canales digitales de difusión.
- **CÁMARAS / SALAS / CAPACIDAD/ PUNTOS DE CULTURA:** Datos de infraestructura, aforo y equipamiento.

Claramente, el archivo contiene muchísima información administrativa que no resulta útil para la dinámica del juego. En la siguiente sección se detalla el proceso de filtrado y descarte de las columnas sobrantes para trabajar y empezar a normalizar únicamente con los datos esenciales.

3. Análisis inicial de los datos.

Con el dataset elegido, antes de iniciar el proceso de normalización, vamos a hacer una limpieza de los datos, de 30 columnas, vamos a quedarnos solo con 6 fundamentales:

- **fid:** Nos sirve como identificador único de cada lugar (ID).
- **ESTABLECIMIENTO:** El nombre del lugar para mostrarlo en el juego cuando el usuario adivine o para listar opciones.
- **FUNCIÓN PRINCIPAL:** La categoría (Museo, Teatro, Bar, etc.) que se convertirá en la tabla de categorías.
- **BARRIO:** El barrio porteño, que se usará para la tabla de barrios y para poder filtrar o dar pistas.
- **LATITUD:** Coordenada Y para ubicar el punto en el mapa.
- **LONGITUD:** Coordenada X para ubicar el punto en el mapa.

Para conseguir este dataset un poco más limpio, vamos a hacer un simple script de Python utilizando pandas.

filtrarcsv.py > ...

```python
import pandas as pd

df = pd.read_csv('espacios-culturales.csv')

columnas_utiles = [
    'fid',
    'ESTABLECIMIENTO',
    'FUNCION_PRINCIPAL',
    'BARRIO',
    'LATITUD',
    'LONGITUD'
]

df_limpio = df[columnas_utiles]

df_limpio.to_csv('espacios-culturales-limpio.csv', index=False)
```

Con las columnas ya filtradas, tocan las filas. También, podemos utilizar Pandas para poder ver todas las categorías que tenemos y cuántas entradas tiene:

```python
import pandas as pd

df = pd.read_csv('espacios-culturales-limpio.csv')
print(df['FUNCION_PRINCIPAL'].value_counts())
```

```
FUNCION_PRINCIPAL
BIBLIOTECA                        619
CENTRO CULTURAL                   436
LIBRERIA                          371
SALA DE TEATRO                    311
MONUMENTOS Y LUGARES HISTORICOS   304
BAR                               253
MUSEO                             176
ESPACIO DE FORMACION              167
GALERIA DE ARTE                   165
DISQUERIA                          62
CALESITA                           49
CLUB DE MUSICA EN VIVO             48
ESPACIO FERIAL                     39
ANFITEATRO                         29
SALA DE CINE                       22
CLUB DE MUSICA EN VIVO - NUEVO      2
```

Varias de estas no queremos que sean posibilidades dentro del juego, entonces vamos a filtrar de la misma forma que filtramos las columnas, para quedarnos entre el rango

de 500 y 1000 muestras, así que decidimos quedarnos con: Museos; Monumentos y Lugares históricos; Salas de Teatro.

filtrarcsv.py > ...

```python
import pandas as pd

df = pd.read_csv('espacios-culturales.csv')


columnas_utiles = [
    'fid',
    'ESTABLECIMIENTO',
    'FUNCION_PRINCIPAL',
    'BARRIO',
    'LATITUD',
    'LONGITUD'
]

df_filtrado = df[columnas_utiles]

categorias_elegidas = [
    'SALA DE TEATRO',
    'MONUMENTOS Y LUGARES HISTORICOS',
    'MUSEO',
]

df_final = df_filtrado[df_filtrado['FUNCION_PRINCIPAL'].isin(categorias_elegidas)]

df_final.to_csv('espacios-culturales-limpio.csv', index=False)

print(f"Total registros originales: {len(df)}")
print(f"Total registros limpios retenidos: {len(df_final)}")

```

```
Total registros originales: 3053
Total registros limpios retenidos: 791
```

Y con eso queda el CSV listo para utilizar como base no relacional.

4. Proceso de normalización.

Antes de normalizar debemos plantearnos ciertas preguntas y tener en cuenta ciertas anomalías

Columnas iniciales del dataset: id_registro, username, email, password, nivel_usuario, id_partida, fecha_inicio, estado_partida , puntaje_total, numero_ronda, nombre_lugar, latitud_adivinada, longitud_adivinada, distancia_metros, puntaje_obtenido

¿Qué representa una fila en la base de datos? Una fila representa un intento/jugada de un usuario en una ronda específica.

¿Cuál es la clave primaria/candidata? Identificar la clave candidata, en este caso es Id_registro

¿Existen dependencias funcionales X → Y?

Username → email, password.

id_partida → username, fecha_inicio, estado_partida,puntaje_total.

id_registro→ numero_ronda, nombre_lugar, latitud_adivinada, longitud_adivinada, distancia_metros, puntaje_obtenido.

Las anomalías que podemos ver son:

Redundancia: Repetir los datos del usuario (email, password, nivel_usuario) y de la partida (fecha_inicio, estado_partida) se repiten en cada ronda disputada

Anomalía de actualización: Si el jugador cambia su usuario o password hay que modificar múltiples filas, si una falla el sistema queda inconsistente.

Anomalía de Inserción: No se puede dar de alta un nuevo usuario si todavía no ha jugado una partida o ronda.

Anomalía de Eliminación: Si se borra el historial de rondas de un jugador, se elimina accidentalmente la partida y la existencia misma del usuario.

Luego de reconocer las anomalías y ver los puntos principales empezamos a normalizar, en la normalización que haremos llegaremos a la 3er forma donde

1FN Elimina: Valores no atómicos / grupos repetitivos:

-1FN donde prioriza los valores atómicos, que dentro de los atributos no haya listas ni colecciones. Asumiendo que las coordenadas latitud_adivinada y longitud_adivinada se guardan en columnas numéricas individuales y no como un par ordenado texto "-34.60,-58.39", la tabla cumple con la 1FN mediante la clave id_registro o mediante la clave compuesta (id_partida, numero_ronda)

2FN Elimina: Dependencias parciales

-2FN Prioriza la eliminación de dependencias parciales, ningún atributo no primo depende de una parte de la clave compuesta, solo aplica con claves compuestas. Al tomar como candidata la clave compuesta, detectamos dependencias parciales, los atributos `fecha_inicio`, `estado_partida`, `puntaje_total` y `username` dependen exclusivamente de `id_partida`, para eliminar esas dependencias separamos los datos para eliminar dependencias. obteniendo 2 entidades **USUARIO_PARTIDA** (id_partida [PK], username, email, password, nivel_usuario, fecha_inicio, estado_partida, puntaje_total) y **RONDA** (id_ronda [PK], id_partida [FK], numero_ronda, nombre_lugar, latitud_adivinada, longitud_adivinada, distancia_metros, puntaje_obtenido)

3FN Elimina: Dependencias transitivas

-3FN Prioriza que los atributos no clave deben depender de la clave, No deben existir dependencias transitivas entre atributos no clave ($X \rightarrow Y$ donde X debe ser superclave o Y un atributo primo) es decir no deben existir cadenas ($X \rightarrow Y \rightarrow Z$). Encontramos una dependencia transitiva en Usuario(id_partida → username → (email,password, nivel_usuario)), también vemos 2 atributos que se podrían modificar, el primero es nivel_usuario el cual se puede calcular en tiempo de ejecución a partir del historial o hasta innecesario para el dominio del juego por lo que se elimina el atributo, el siguiente atributo a modificar es nombre_lugar el cual se reemplaza por id_lugar para evitar guardar texto repetido. eliminando las dependencias transitivas y separar los hechos puros del negocio, el esquema relacional queda dividido en las 3 tablas .

USUARIO

- id_usuario (PK, AUTOINCREMENTAL)
- username (VARCHAR)
- email (VARCHAR)
- password (VARCHAR)

PARTIDA

- id_partida (PK, AUTOINCREMENTAL)
- id_usuario (FK → USUARIO.id_usuario)
- puntaje_total (INT)
- fecha_inicio (TIMESTAMP)
- estado (VARCHAR)

RONDA

- id_ronda (PK, AUTOINCREMENTAL)
- id_partida (FK → PARTIDA.id_partida)
- numero_ronda (INT)
- id_lugar (VARCHAR, referencia a FID de la base en MongoDB)
- latitud_adivinada (DECIMAL)
- longitud_adivinada (DECIMAL)
- distancia_metros (DECIMAL)
- puntaje_obtenido (INT)

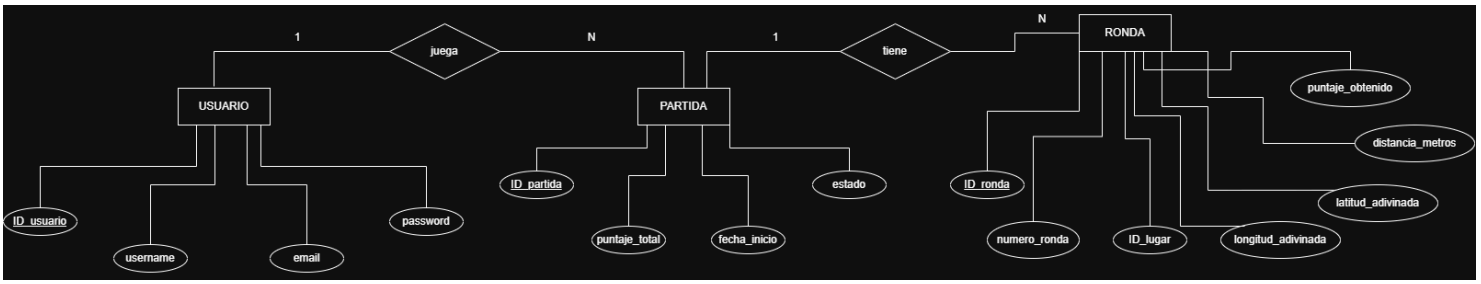
Conclusión: El modelo resultante se encuentra en 3FN. Se eliminan las redundancias, se garantiza las restricciones de integridad referencial mediante claves foráneas y se previnieron las anomalías de inserción, modificación y eliminación.

5. Diagrama Entidad-Relación.

El Diagrama Entidad-Relación resultante está compuesto por tres entidades: USUARIO, PARTIDA y RONDA. La entidad USUARIO representa a cada jugador registrado en el sistema, e incluye los atributos username, email y password, identificados por id_usuario como clave primaria. La entidad PARTIDA representa cada juego iniciado por un usuario, con los atributos puntaje_total, fecha_inicio y estado, identificados por id_partida. Por último, la entidad RONDA representa cada una de las instancias dentro de una partida, con los atributos numero_ronda, id_lugar,

longitud_adivinada, latitud_adivinada, distancia_metros y puntaje_obtenido, identificados por id_ronda. El atributo id_lugar referencia al espacio cultural almacenado en la base de datos no relacional, por lo que se modela como un atributo común y no como una relación entre entidades.

Se establecen dos relaciones entre las entidades: juega, que vincula a cada usuario con sus partidas, y tiene, que vincula a cada partida con sus rondas. Ambas relaciones presentan cardinalidad 1:N: un usuario puede jugar muchas partidas, pero cada partida pertenece a un único usuario; de la misma manera, una partida se compone de varias rondas, pero cada ronda pertenece a una única partida.



6. Modelo Relacional.



A partir del DER presentado en el punto anterior, procedimos a la transformación al Modelo Relacional implementando las tablas **USUARIO**, **PARTIDA** y **RONDA**.

Cada tabla cuenta con su **PRIMARY KEY**, que corresponde al atributo subrayado en el diagrama (**id_usuario**, **id_partida** e **id_ronda**). Para su implementación en PostgreSQL, los IDs se definen mediante el tipo **SERIAL** para generar números enteros autoincrementales y así identificar unívocamente a cada registro en su respectiva tabla. Para las cadenas de texto utilizamos **VARCHAR**, ajustando el

tamaño a cada caso: **VARCHAR(50)** para el username, **VARCHAR(100)** para el email y **VARCHAR(255)** para el password (permitiendo almacenar hashes seguros sin truncar datos). El atributo **id_lugar** dentro de **RONDA** se define como **VARCHAR(50)** ya que almacena el identificador del espacio cultural proveniente de la base no relacional (MongoDB), actuando como un dato plano sin vínculo de clave foránea física en este motor.

Por su parte, los puntajes y el número de ronda se modelan como números enteros (**INT**), mientras que la **fecha_inicio** utiliza **TIMESTAMP** para registrar fecha y hora exacta. En cuanto a las coordenadas, **latitud_adivinada** y **longitud_adivinada** se definen como **DECIMAL(10, 8)** y **DECIMAL(11, 8)** respectivamente, garantizando precisión exacta de punto fijo para almacenar las ubicaciones geográficas marcadas por el usuario dentro de sus rangos válidos (-90 a 90 y -180 a 180) sin sufrir desfasajes por redondeo. La **distancia_metros** se define como **DECIMAL(10, 2)** para registrar con exactitud la distancia calculada respecto al punto real.

Por otra parte, indicamos la restricción **NOT NULL** en las columnas para asegurar la completitud de la información, sumando la restricción **UNIQUE** en el **username** y en el **email** para impedir registros duplicados.

Para plasmar las relaciones **1:N** que venían del modelo conceptual, aplicamos la regla de pasaje relacional: la **PRIMARY KEY** de **USUARIO** se registra como **FOREIGN KEY** (**id_usuario**) dentro de **PARTIDA** y, a su vez la **PRIMARY KEY** de **PARTIDA** se registra como **FOREIGN KEY** (**id_partida**) dentro de **RONDA**. De este modo, un único usuario puede disputar múltiples partidas y cada partida pertenecer exclusivamente a ese usuario, al tiempo que una partida puede componerse de diversas rondas pero cada ronda corresponder a una sola partida específica, aplicando la restricción **ON DELETE RESTRICT** para preservar el historial de juego y evitar inconsistencias en cascada.

A continuación presentamos el **script DDL** para implementar la base de datos en PostgreSQL. Para crearlas, respetamos estrictamente el orden de dependencias: primero generamos la tabla maestra independiente (**USUARIO**), luego creamos la tabla intermedia (**PARTIDA**) que referencia al usuario, y por último creamos **RONDA**, que depende directamente de la partida.

Además de las claves y tipos de datos mencionados, agregamos restricciones clave para cuidar la integridad de la información: por un lado, **ON DELETE RESTRICT** y **ON UPDATE CASCADE** en las claves foráneas para preservar el historial de jugadas y evitar borrar usuarios o partidas activas; por otro lado, incluimos restricciones **CHECK** en **latitud_adivinada** y **longitud_adivinada** para validar que las coordenadas ingresadas por el jugador no se salgan de los rangos reales permitidos (-90 a 90 y -180 a 180), como así también validaciones de que los puntajes, número de ronda y distancia en metros no admitan valores negativos.

```sql
-- Eliminamos tablas en orden inverso a sus dependencias
DROP TABLE IF EXISTS RONDA;
DROP TABLE IF EXISTS PARTIDA;
DROP TABLE IF EXISTS USUARIO;

-- 1. Creamos la tabla independiente: USUARIO
CREATE TABLE USUARIO (
    id_usuario SERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL
);

-- 2. Creamos la tabla dependiente de USUARIO: PARTIDA
CREATE TABLE PARTIDA (
    id_partida SERIAL PRIMARY KEY,
    id_usuario INT NOT NULL,
    puntaje_total INT NOT NULL DEFAULT 0,
    fecha_inicio TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(20) NOT NULL,

    -- Restricción de Clave Foránea
    CONSTRAINT fk_partida_usuario
        FOREIGN KEY (id_usuario)
        REFERENCES USUARIO (id_usuario)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    -- Restricción de Dominio
    CONSTRAINT chk_puntaje_total_positivo
        CHECK (puntaje_total >= 0)
);

-- 3. Creamos la tabla dependiente de PARTIDA: RONDA
CREATE TABLE RONDA (
    id_ronda SERIAL PRIMARY KEY,
    id_partida INT NOT NULL,
    numero_ronda INT NOT NULL,
    id_lugar VARCHAR(50) NOT NULL, -- identificador en MongoDB
    latitud_adivinada DECIMAL(10, 8) NOT NULL,
    longitud_adivinada DECIMAL(11, 8) NOT NULL,
    distancia_metros DECIMAL(10, 2) NOT NULL,
    puntaje_obtenido INT NOT NULL,

    -- Restricción de Clave Foránea
    CONSTRAINT fk_ronda_partida
        FOREIGN KEY (id_partida)
        REFERENCES PARTIDA (id_partida)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
```

```
    -- Restricciones de Dominio: Coordenadas geográficas válidas
    CONSTRAINT chk_latitud_adivinada_rango
        CHECK (latitud_adivinada BETWEEN -90.0 AND 90.0),

    CONSTRAINT chk_longitud_adivinada_rango
        CHECK (longitud_adivinada BETWEEN -180.0 AND 180.0),

    -- Restricciones de Dominio: Valores no negativos y rondas válidas
    CONSTRAINT chk_numero_ronda_valido
        CHECK (numero_ronda > 0),

    CONSTRAINT chk_distancia_positiva
        CHECK (distancia_metros >= 0.0),

    CONSTRAINT chk_puntaje_obtenido_positivo
        CHECK (puntaje_obtenido >= 0)
);
```

7. Diseño de la solución NoSQL.

Para la solución NoSQL se decidió utilizar **MongoDB**, un motor de base de datos orientado a documentos, aplicado específicamente a la gestión del catálogo geoespacial de espacios culturales de la Ciudad de Buenos Aires.

A diferencia del dominio relacional (donde PostgreSQL gestiona las transacciones de usuarios, partidas y rondas), el catálogo de lugares se beneficia de un modelo NoSQL ya que cada espacio cultural se almacena como un documento autocontenido. Esto permite agrupar sus atributos descriptivos y su geometría espacial sin necesidad de realizar uniones de tablas (*JOINs*).

Se define la base de datos ***bairesguessr_nosql*** con la colección ***puntos_culturales***. Cada documento representa un espacio cultural con la siguiente estructura JSON:

```
{
  "_id": ObjectId("652f1a9b8c3d2e1f0a4b5c6d"),
  "fid": 123,
  "establecimiento": "El Ateneo Grand Splendid",
  "funcion_principal": "BIBLIOTECA",
  "barrio": "RECOLETA",
  "ubicacion": {
    "type": "Point",
    "coordinates": [-58.3924, -34.5960]
  }
}
```

Donde *ubicacion* sigue el estándar universal GeoJSON Point, almacenando las coordenadas geográficas en formato *[longitud, latitud]*.

Para optimizar las consultas espaciales del juego y permitir búsquedas de proximidad en tiempo real, se define un índice de superficie esférica 2dsphere sobre el campo *ubicacion*. Esto permite que el motor resuelva consultas geográficas y selección de puntos aleatorios a alta velocidad sin sobrecargar la capa de aplicación.

8. Justificación de la utilización de NoSQL.

La implementación de una arquitectura de persistencia políglota, donde MongoDB convive activamente con PostgreSQL en el backend, responde a la necesidad de optimizar las lecturas y consultas espaciales de alta frecuencia dentro de la dinámica del juego.

En primer lugar, se seleccionó MongoDB por su soporte nativo para el estándar GeoJSON y la creación de índices espaciales 2dsphere. Dado que la mecánica central del juego requiere obtener coordenadas reales de lugares y resolver consultas de proximidad o filtros geográficos (por ejemplo, obtener puntos dentro de un radio determinado o calcular distancias sobre la curvatura de la tierra), MongoDB ejecuta estas operaciones a nivel de motor de base de datos en milisegundos, evitando realizar cálculos trigonométricos costosos sobre bases relacionales tradicionales.

En segundo lugar, el modelo documental flexible facilita el empaquetado de la información de cada espacio cultural (nombre, barrio, función principal y ubicación) en un solo objeto JSON. Esto se alinea de forma natural con el flujo de datos entre la API REST construida en Spring Boot y el cliente en React, reduciendo la latencia de lectura al eliminar la necesidad de unir múltiples tablas relacionales para mostrar un punto cultural en la interfaz.

Finalmente, esta separación de responsabilidades permite desvincular el catálogo de datos geográficos del modelo de negocio relacional, logrando que MongoDB asuma las lecturas masivas y búsquedas espaciales, mientras PostgreSQL garantiza la

integridad transaccional (ACID) en el registro de usuarios, partidas e historial de puntuaciones.

9. Arquitectura general de la aplicación.

Para el desarrollo de nuestro juego decidimos utilizar una arquitectura cliente-servidor, separando las responsabilidades en tres componentes principales: el frontend, el backend y la capa de persistencia de datos.

El frontend, desarrollado con React, TypeScript y Vite, utiliza Chakra UI para la interfaz visual y React Router para la navegación entre vistas, constituyendo el punto de interacción con el usuario. Integra la librería MapillaryJS para conectarse directamente con la API externa de Mapillary y renderizar el entorno interactivo en 360° a partir del identificador de imagen provisto por el servidor. Asimismo, incorpora la librería Leaflet junto con capas cartográficas de OpenStreetMap para desplegar el mapa interactivo de la Ciudad Autónoma de Buenos Aires, permitiendo al jugador marcar su estimación y visualizar posteriormente tanto la ubicación real como la distancia obtenida y el puntaje de cada ronda.

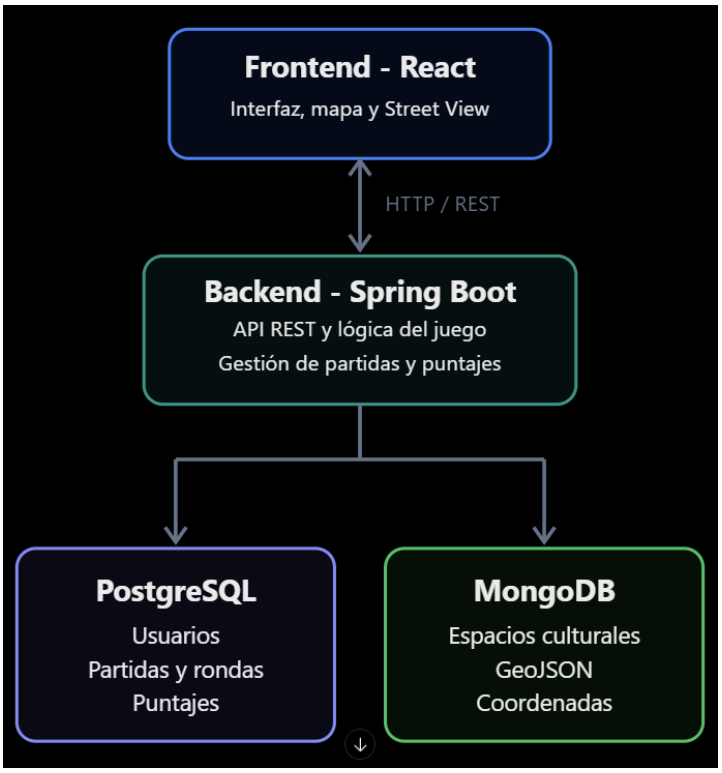
El backend, desarrollado en Kotlin sobre el framework Spring Boot, funciona como intermediario entre el cliente y las bases de datos exponiendo una API REST. Para interactuar con los motores de datos emplea Spring Data JPA en el acceso relacional y Spring Data MongoDB en el documental. A través de sus servicios recibe las solicitudes del cliente, procesa la lógica de negocio y se encarga de la gestión de usuarios y sesiones, la creación de partidas, la selección aleatoria de ubicaciones sin repetición, el registro de rondas, el cálculo geodésico de distancias entre coordenadas y la determinación de los puntajes.

Para la persistencia de datos utilizamos dos motores diferentes orquestados mediante contenedores Docker, asignando a cada uno una responsabilidad específica según la naturaleza de la información. Por un lado, PostgreSQL almacena los datos transaccionales y estructurados del sistema, como los usuarios, las partidas y el detalle de las rondas disputadas, garantizando la integridad referencial mediante claves primarias, claves foráneas y restricciones, y vinculando cada ronda con su escenario

mediante el identificador del documento correspondiente. Por otro lado, MongoDB almacena el catálogo documental de espacios culturales de la Ciudad Autónoma de Buenos Aires, construido a partir de los datos abiertos del GCBA y completado previamente con los identificadores de imágenes de Mapillary. Cada documento modela su ubicación bajo el estándar GeoJSON de tipo Point junto con sus metadatos e incorpora un índice geoespacial 2dsphere para optimizar las consultas espaciales y la selección dinámica de escenarios.

El funcionamiento general comienza cuando el usuario inicia una partida desde el frontend, lo que genera una solicitud HTTP hacia el backend. Este registra la nueva partida en PostgreSQL y consulta MongoDB para obtener aleatoriamente un espacio cultural que no haya sido jugado en esa partida, devolviendo al cliente el identificador de Mapillary para cargar la vista panorámica. Una vez que el jugador selecciona un punto en el mapa de Leaflet, o al cumplirse el tiempo límite de la ronda, el frontend envía las coordenadas elegidas al backend. Allí se recuperan las coordenadas reales del documento en MongoDB, se calcula la distancia en metros y se asigna el puntaje correspondiente, persistiendo finalmente el resultado en PostgreSQL antes de devolverlo al frontend para mostrarlo en pantalla.

De esta manera, conseguimos una arquitectura desacoplada donde cada capa tiene una responsabilidad definida y un despliegue reproducible gracias a la contenerización de las bases de datos. El frontend se ocupa de la experiencia visual y geográfica del usuario, el backend centraliza la lógica del juego y la persistencia políglota aprovecha tanto la consistencia relacional de PostgreSQL como la flexibilidad espacial de MongoDB, facilitando el mantenimiento y la evolución del proyecto sin necesidad de rediseñar toda la aplicación.



10. Explicación del backend polimórfico.

Para el desarrollo del backend de BairesGuessr, procedimos a implementar una arquitectura en capas utilizando Kotlin y Spring Boot 3, diseñada para operar de manera simultánea sobre los dos motores de persistencia definidos en los modelos anteriores: PostgreSQL para los datos transaccionales y MongoDB para el catálogo documental de espacios culturales.

El sistema se estructuró separando estrictamente la interfaz de comunicación, la lógica de negocio y el acceso a los datos mediante interfaces comunes y repositorios desacoplados. De este modo, los controladores REST (AuthController y JuegoController) exponen los endpoints hacia el frontend sin conocer qué tecnología de base de datos resuelve cada operación, delegando toda la ejecución en la capa de servicios (JuegoService), que actúa como punto único de coordinación entre ambos motores.

Para concretar la comunicación con las bases de datos desde una misma aplicación, definimos paquetes de persistencia independientes (repositories.postgres y repositories.mongo) bajo el patrón Repository. A través de este diseño, el servicio

invoca interfaces declarativas (UsuarioRepository, PartidaRepository, RondaRepository mediante Spring Data JPA, y EspacioCulturalRepository mediante Spring Data MongoDB), delegando la traducción de consultas SQL y BSON a los adaptadores configurados en application.yml sin acoplar las reglas del juego a un motor específico.

Respecto a las operaciones del sistema, implementamos las acciones fundamentales de alta, consulta individual, listado y actualización repartidas de acuerdo con la naturaleza de cada entidad:

Gestión de usuarios y partidas (PostgreSQL): A través del servicio y los repositorios relacionales, implementamos el alta de nuevos usuarios (/api/auth/register), la inicialización de partidas (/api/partidas) y la consulta del estado y puntaje acumulado de una partida específica junto con el listado de sus rondas jugadas (GET /api/partidas/{idPartida}).

Gestión del catálogo geográfico (MongoDB): Mediante el repositorio documental, realizamos consultas individuales por identificador y recuperación de documentos geoespaciales desde la colección de espacios culturales para obtener tanto las pistas visuales/descriptivas como las coordenadas reales de cada establecimiento.

Por otra parte, para evitar que el backend se limite a búsquedas simples por clave primaria, desarrollamos una operación compuesta de alta complejidad en el flujo central de juego (POST /api/partidas/{idPartida}/rondas/nueva y POST /api/partidas/{idPartida}/rondas/{idRonda}/adivinar), la cual relaciona información de ambos motores en tiempo real:

Selección sin repetición: Al solicitar una nueva ronda, el servicio consulta en PostgreSQL todos los id_lugar ya jugados en esa partida y cruza esa lista contra la colección de MongoDB para seleccionar un espacio cultural inédito, persistiendo el nuevo registro de ronda en la base relacional.

Cálculo geodésico y liquidación de puntaje: Al recibir las coordenadas marcadas por el usuario en el mapa (latitudAdivinada y longitudAdivinada), el servicio recupera de PostgreSQL el registro de la ronda activa y extrae su id_lugar. Con ese identificador plano, consulta el documento correspondiente en MongoDB para obtener las

coordenadas exactas del lugar (latitudReal y longitudReal). Luego, aplica en memoria la fórmula trigonométrica de Haversine para calcular la distancia real en metros entre ambos puntos, asigna el puntaje proporcional (entre 0 y 5000 puntos), actualiza la fila de RONDA y acumula el total en la tabla PARTIDA dentro de una misma transacción relacional, controlando además el cierre automático al completar la quinta ronda.

De esta forma, el backend garantiza la integridad de los datos mediante validaciones de entrada en los DTOs (@Valid), centraliza el acceso a ambas bases de datos impedido directamente desde el cliente y articula la persistencia relacional y documental dentro de un único flujo de aplicación.

11. Tecnologías utilizadas.

Para la construcción integral de BairesGuessr se seleccionó un ecosistema de tecnologías modernas, desacopladas y orientadas a facilitar tanto el modelado de datos relacional y documental como la experiencia geográfica interactiva del usuario, Podemos separar las tecnologías en 4 tipos.

A) Frontend (Interfaz de usuario y visualización geográfica):

React y TypeScript: Biblioteca principal para la construcción de componentes modulares y reactivos con tipado estático fuerte.

Vite: Entorno de desarrollo y empaquetador de módulos de alta velocidad para la compilación del cliente web.

Chakra UI: Biblioteca de componentes visuales utilizada para diseñar las pantallas de autenticación, menú principal, barra de estado de partida, tablas de historial y tarjetas de perfil.

MapillaryJS: SDK oficial integrado para renderizar las vistas panorámicas interactivas en 360° a nivel de calle mediante el consumo de la API de Mapillary.

Leaflet y React-Leaflet (con OpenStreetMap): Biblioteca cartográfica utilizada para desplegar el mini mapa interactivo de la Ciudad Autónoma de Buenos Aires, capturar las coordenadas seleccionadas por el jugador y trazar visualmente la distancia entre el punto estimado y la ubicación real.

Axios: Cliente HTTP empleado para consumir los endpoints REST del servidor backend.

B) Backend (Lógica de negocio y API REST):

Kotlin: Lenguaje principal del servidor por su concisión, seguridad ante nulos (null-safety) e interoperabilidad total con el ecosistema JVM.

Spring Boot: Framework base para exponer la API RESTful (AuthController, JuegoController y LeaderboardController), gestionar inyección de dependencias y validar DTOs de entrada.

Gradle: Herramienta de automatización de compilación y gestión de dependencias del proyecto backend.

C) Persistencia de Datos e Infraestructura:

PostgreSQL: Sistema gestor de bases de datos relacional encargado de garantizar la integridad referencial y las propiedades ACID sobre las entidades USUARIO, PARTIDA y RONDA.

MongoDB: Base de datos orientada a documentos utilizada para almacenar el catálogo de espacios culturales de CABA bajo el estándar GeoJSON (Point).

Docker y Docker Compose: Plataforma de contenerización empleada para orquestar ambos motores de bases de datos de manera aislada y ejecutar automáticamente los scripts de inicialización (script.sql e init-mongo.js) en cualquier sistema operativo.

D) Procesamiento de Datos y Control de Versiones:

Python y Pandas: Herramientas utilizadas en la etapa previa de análisis exploratorio, limpieza de columnas y filtrado del dataset abierto del Gobierno de la Ciudad de Buenos Aires.

Git y GitHub: Control de versiones distribuido mediante flujo de ramas (main, develop y ramas feature) para el trabajo colaborativo del equipo.

12. Instrucciones para ejecutar el proyecto.

En primer lugar, se debe clonar el repositorio desde el siguiente enlace: https://github.com/LeanSChavez/BairesGuessr. Una vez descargado el proyecto, es necesario configurar la clave de acceso para la visualización de entornos en 360°; para ello, se debe ingresar a la carpeta **/frontend** y crear un archivo llamado **.env** que contenga la variable de entorno con el token de Mapillary siguiendo el formato **VITE_MAPILLARY_TOKEN=MLY|(su_token)**, el cual puede generarse desde la plataforma de desarrolladores de Mapillary o solicitarse a los administradores del repositorio.

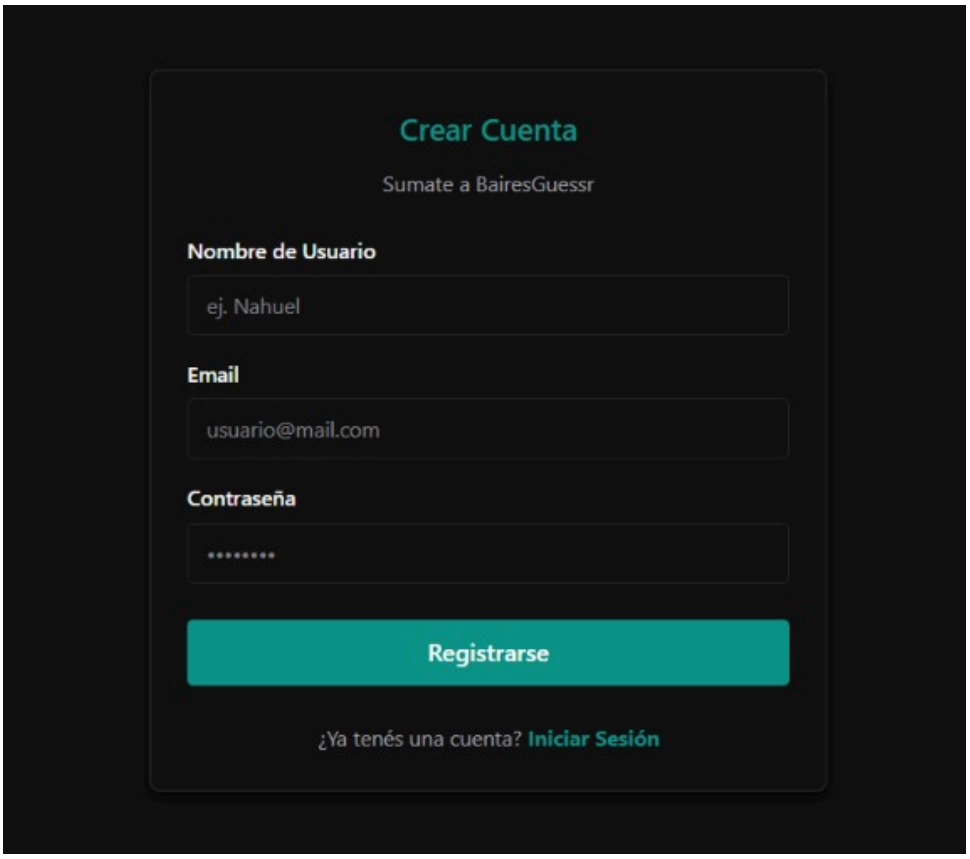
Con la configuración inicial resuelta, el siguiente paso consiste en desplegar la infraestructura de persistencia ejecutando el comando **docker compose up -d** desde el directorio raíz del proyecto. El parámetro **-d** (*detached mode*) permite iniciar los contenedores en segundo plano, dejando la terminal libre para continuar trabajando; en este primer arranque se construyen automáticamente las instancias de PostgreSQL y MongoDB, ejecutando los scripts de inicialización que crean el esquema relacional y cargan el catálogo de espacios culturales junto con su índice geoespacial.

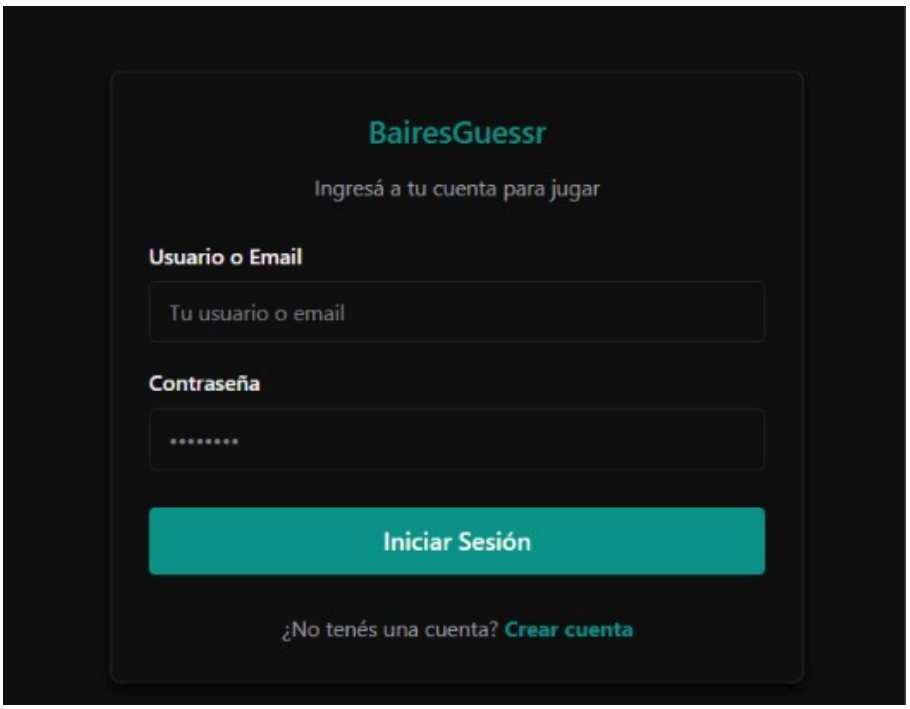
Finalmente, con las bases de datos activas, se debe iniciar la capa de lógica de negocio levantando el backend mediante un entorno de desarrollo como IntelliJ IDEA o Visual Studio Code (o por consola mediante Gradle), ejecutando el archivo principal **BairesGuessrApplication.kt** ubicado dentro de **/backend**. En paralelo, para poner en marcha la interfaz web, se debe abrir una terminal posicionada en la carpeta **/frontend**, ejecutar **pnpm install** para descargar las dependencias del proyecto y luego iniciar el servidor de desarrollo mediante **pnpm dev**. Con todos los servicios en ejecución, basta con acceder desde el navegador a la dirección local provista por Vite para ingresar a la pantalla de inicio de sesión, registrar una cuenta y comenzar a jugar.

13. Ejemplos de funcionamiento.

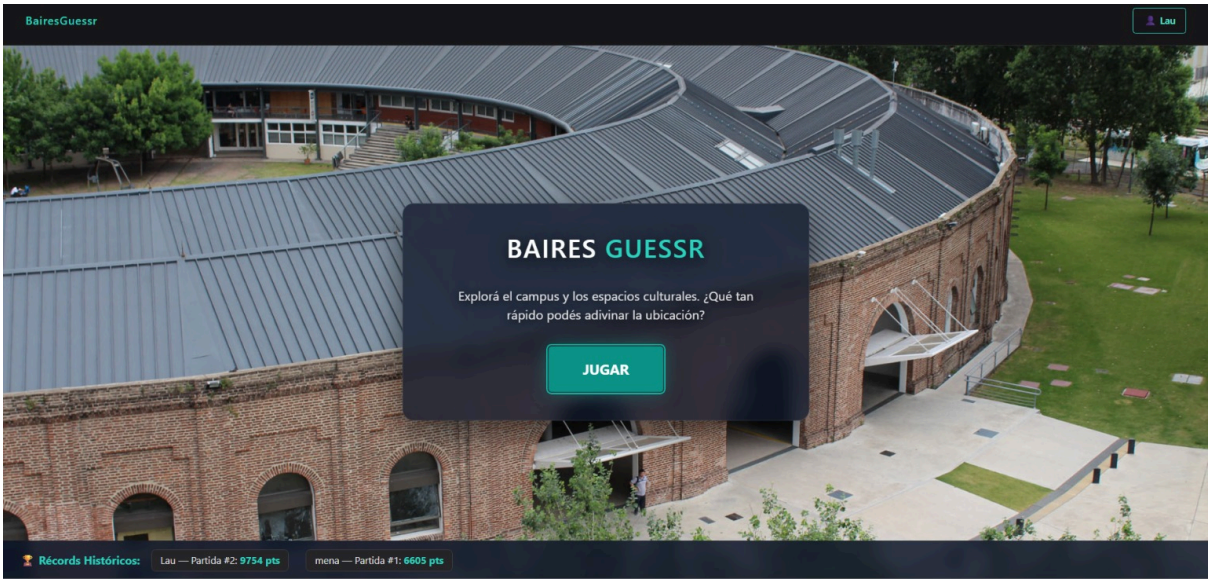
A continuación se ilustra el flujo completo de ejecución de BairesGuessr, evidenciando la interacción entre la interfaz web, el servidor backend y ambos motores de bases de datos:

1. Registro e inicio de sesión de usuarios (Persistencia en PostgreSQL): Al ingresar a la aplicación, el usuario puede crear una cuenta nueva (/register) o iniciar sesión (/login) con sus credenciales. El backend valida los datos recibidos, encripta la contraseña y persiste el registro en la tabla USUARIO de PostgreSQL.



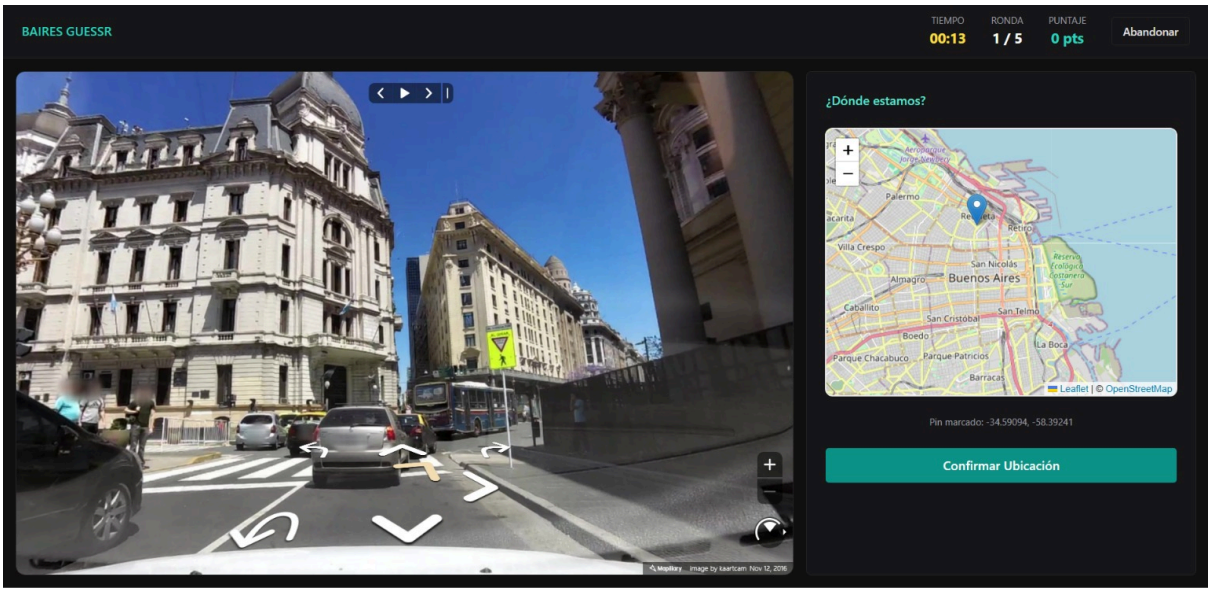


2. Menú principal y Récords Históricos: Una vez autenticado, el jugador accede al menú principal (/menu), donde se visualiza en la barra inferior el ranking dinámico (Récords Históricos) consultado en tiempo real desde el endpoint GET /api/leaderboard, reflejando los mejores puntajes de partidas finalizadas almacenados en PostgreSQL

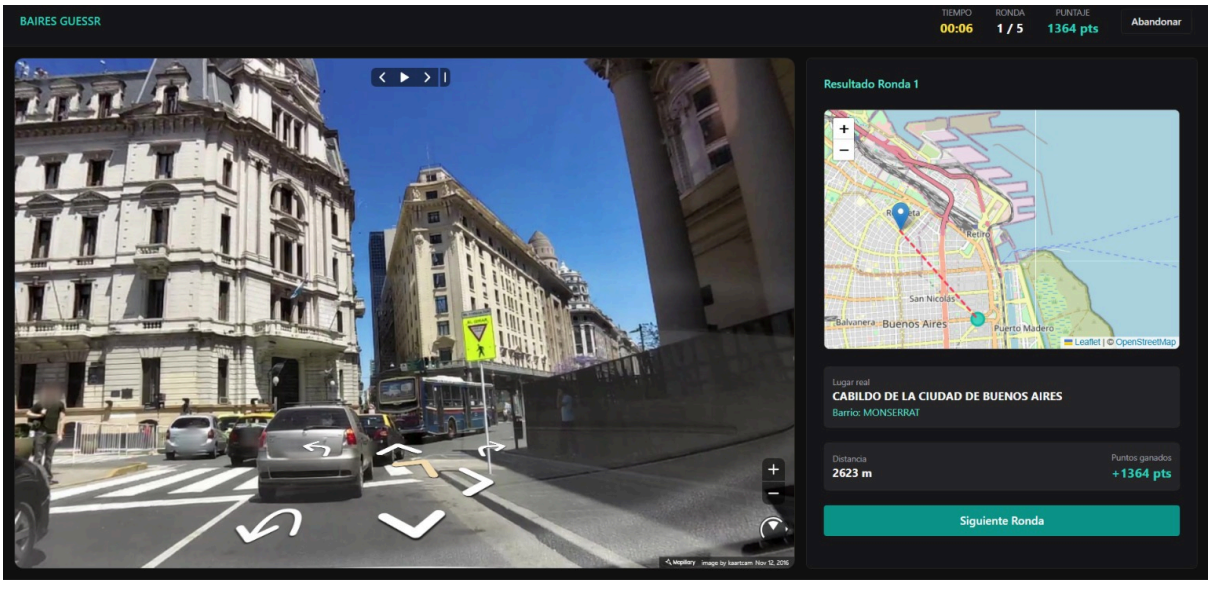


3. Desarrollo de una partida y exploración en 360°: Al presionar JUGAR, el backend crea un nuevo registro en PARTIDA (estado = 'EN_PROGRESO') y selecciona aleatoriamente un espacio cultural desde MongoDB que no haya sido jugado en esa sesión. En pantalla se despliega el visor interactivo de Mapillary en

360° junto con la barra superior de estado (ronda actual 1/5, temporizador y puntaje acumulado) y el minimapa de Leaflet donde el usuario marca su estimación.

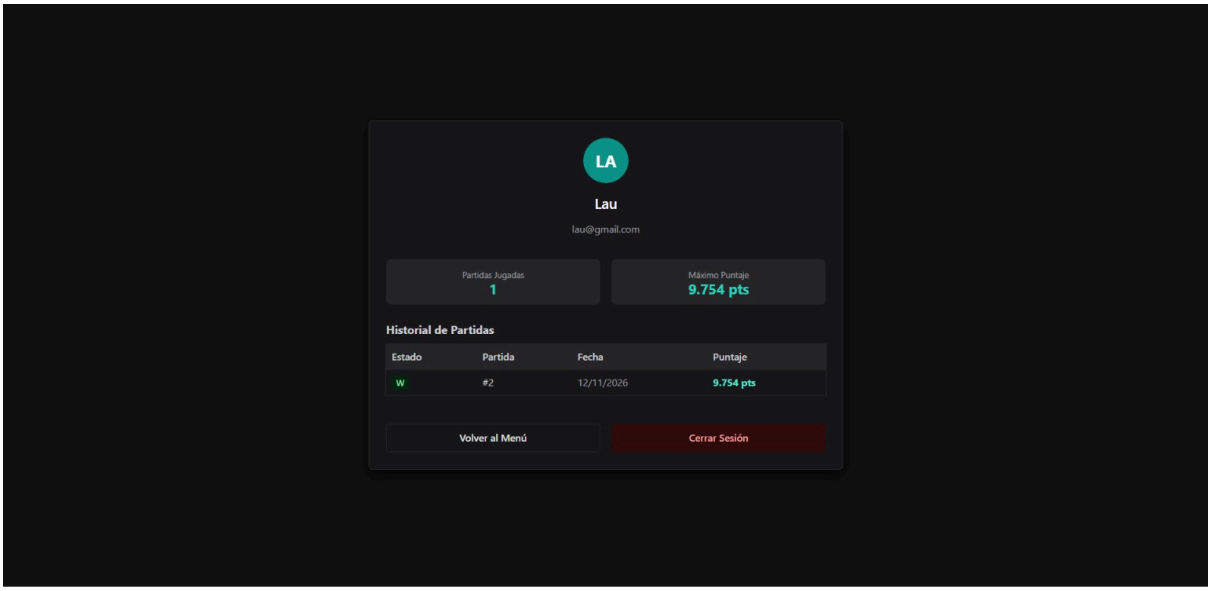


4. Resolución de ronda y cálculo geodésico: Al confirmar la ubicación, el servidor recupera las coordenadas reales del documento en MongoDB, calcula la distancia y asigna el puntaje proporcional. En la interfaz se traza una línea punteada entre la ubicación marcada y la ubicación real (por ejemplo, Cabildo de la Ciudad de Buenos Aires a 2623 m con +1364 pts,) persistiendo el detalle en la tabla RONDA.

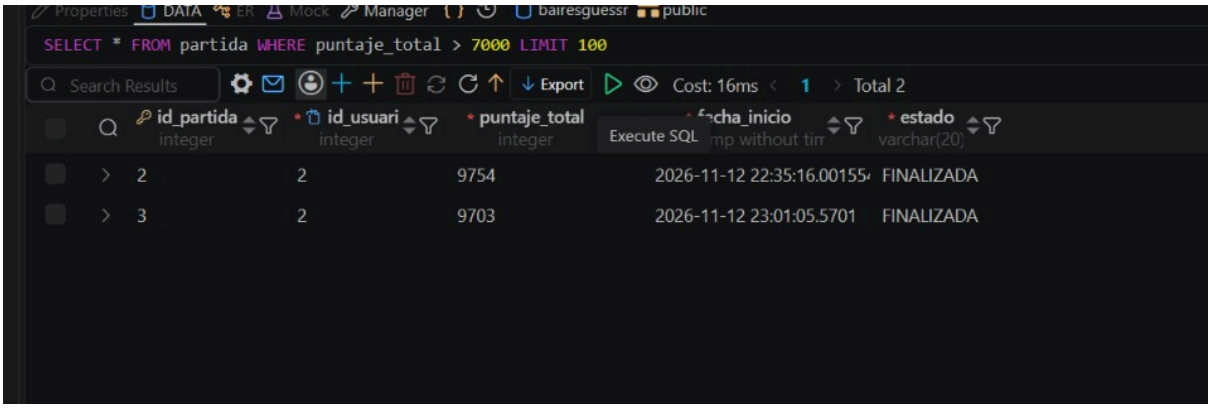


5. Historial y estadísticas en el Perfil de Usuario: Al completar las 5 rondas, la partida pasa automáticamente a estado FINALIZADA y se actualiza su puntaje_total. Desde la vista de perfil (/profile), el usuario puede consultar sus métricas acumuladas

(Partidas Jugadas, Máximo Puntaje: 9.754 pts) y el detalle de su historial obtenido desde GET /api/partidas/usuario/{idUsuario}.



6. Verificación de persistencia y consultas sobre PostgreSQL: Finalmente, mediante consultas SQL ejecutadas directamente sobre la instancia de PostgreSQL (SELECT * FROM partida WHERE puntaje_total > 7000;), se verifica que las partidas finalizadas por distintos usuarios quedaron correctamente almacenadas con sus identificadores, puntajes totales y marcas de tiempo (timestamp).



14. Comparación entre ambas implementaciones.

La coexistencia de PostgreSQL y MongoDB dentro de BairesGuessr permite contrastar en la práctica cómo responde cada paradigma frente a los requerimientos específicos de un mismo dominio de aplicación. A diferencia de una arquitectura monolítica respaldada por un único motor, la implementación de ambos modelos evidenció diferencias sustanciales en el diseño del esquema, el tratamiento de la integridad de los datos, la flexibilidad evolutiva y la resolución de consultas.

Esquema, estructura y evolución de los datos

En PostgreSQL, el modelado exige una definición estricta y anticipada mediante sentencias DDL (schema-on-write). Cada tabla (USUARIO, PARTIDA, RONDA) impone tipos de datos fijos, longitudes acotadas (VARCHAR(50), DECIMAL(10, 8)) y obligatoriedad de campos (NOT NULL). Si bien esta rigidez demanda un proceso previo de normalización hasta Tercera Forma Normal (3FN) y dificulta modificaciones estructurales rápidas, resulta indispensable para garantizar que las métricas de puntaje, las fechas de inicio y las coordenadas enviadas por el usuario mantengan exactitud matemática y consistencia absoluta.

Por el contrario, MongoDB opera bajo un enfoque flexible (schema-on-read), almacenando los espacios culturales como documentos JSON/BSON autocontenidos. Esta característica resultó determinante al trabajar con el dataset abierto gubernamental: muchos establecimientos culturales presentan metadatos heterogéneos (algunos poseen dirección exacta, teléfono, sitio web o múltiples categorías, mientras que otros carecen de esos atributos). En un modelo relacional puro, esto hubiera obligado a crear tablas auxiliares de normalización o a tolerar numerosas columnas con valores NULL; en MongoDB, cada documento almacena únicamente los campos disponibles sin penalizar el espacio ni requerir migraciones de esquema ante la incorporación de nuevos atributos.

Integridad referencial vs. Desacoplamiento documental

Una de las diferencias más marcadas entre ambas implementaciones radica en la gestión de las relaciones y restricciones:

Integridad fuerte (PostgreSQL): El motor relacional protege la coherencia histórica del juego mediante claves foráneas (FOREIGN KEY), restricciones de dominio (CHECK sobre rangos de latitud y longitud) y políticas de borrado (ON DELETE RESTRICT). Es físicamente imposible que exista una ronda huérfana sin una partida asociada o una partida sin un usuario válido, delegando esa responsabilidad directamente en el motor de base de datos.

Integridad a nivel de aplicación (MongoDB): Al carecer de claves foráneas nativas y de un vínculo físico entre motores, la relación entre la tabla RONDA (Postgres) y la colección espacios_culturales (Mongo) se resuelve mediante el identificador plano id_lugar (VARCHAR(50)). En este esquema, la validación de existencia del documento y la consistencia referencial recaen enteramente en la lógica de la capa de servicio (JuegoService) desarrollada en Kotlin, lo que otorga mayor velocidad de lectura y desacoplamiento, pero exige controles programáticos explícitos para evitar referencias rotas.

Rendimiento, transacciones y naturaleza de las consultas

En cuanto al comportamiento operativo, PostgreSQL demostró su fortaleza en las operaciones de escritura concurrente y actualización de estado. Durante la liquidación de cada ronda, el motor asegura propiedades ACID mediante transacciones atómicas: al procesar una adivinanza, se actualiza el resultado de la ronda y se incrementa el puntaje acumulado de la partida en una misma unidad lógica de trabajo, evitando condiciones de carrera o estados inconsistentes si ocurriera un fallo intermedio.

En cambio, MongoDB se destacó en la lectura rápida de catálogos y el filtrado de documentos sin necesidad de ejecutar operaciones JOIN costosas. Al recuperar un espacio cultural para iniciar una ronda o verificar sus coordenadas reales, toda la información del establecimiento (nombre, barrio, categoría, pistas y ubicación geográfica) se obtiene en una única lectura de documento.

En síntesis, la comparación práctica confirma que ninguno de los dos motores reemplaza de manera óptima al otro dentro de este dominio: mientras PostgreSQL aporta el rigor estructural y transaccional necesario para sostener las reglas del juego y el historial de los usuarios, MongoDB provee la agilidad y flexibilidad documental ideales para gestionar el catálogo geográfico de la ciudad.

15. Problemas encontrados y decisiones tomadas.

Uno de los primeros inconvenientes surgió en la etapa inicial de diseño, con el mal planteamiento de qué información debía residir en cada base de datos. Tras la revisión del checkpoint, quedamos en que la mejor opción de diseño, era dejar en el motor SQL todo lo relacionado a la partida y al usuario, mientras que en el motor NOSQL, viviría el catálogo de espacio en formato JSON para aprovechar las capacidades del motor para consultas geográficas.

A partir de esta separación entre dos motores distintos, enfrentamos el desafío de vincular las rondas almacenadas en PostgreSQL con los espacios culturales guardados en MongoDB, ya que no es posible definir claves foráneas nativas entre bases de datos diferentes. Para resolverlo sin acoplar rígidamente ambos sistemas, decidimos almacenar en la tabla de rondas el identificador único del documento de MongoDB como una cadena de texto, delegando en la lógica del backend la tarea de cruzar ambos registros y calcular la distancia entre las coordenadas elegidas por el jugador y la ubicación real del lugar.

Otro problema importante estuvo relacionado con la preparación de los datos geográficos, dado que el listado oficial de espacios culturales del Gobierno de la Ciudad contaba con coordenadas y descripciones, pero no incluía referencias a vistas panorámicas a nivel de calle, y varias ubicaciones no tenían cobertura visual cercana. Para solucionarlo, implementamos un proceso previo de limpieza y enriquecimiento utilizando la API de Mapillary, asociando cada establecimiento con el identificador de una imagen 360° real dentro de un radio cercano y filtrando aquellos lugares que no contaban con imágenes disponibles.

Finalmente, a medida que el proyecto creció, levantar el entorno de desarrollo se volvió complejo y propenso a errores entre distintos sistemas operativos como Linux y Windows, ya que exigía ejecutar scripts manualmente y configurar cada base de datos por separado. Para garantizar que cualquier integrante del equipo pudiera ejecutar la aplicación de forma sencilla y reproducible, fuimos evolucionando la configuración de Docker Compose para orquestar PostgreSQL y MongoDB de manera independiente y automatizar tanto la creación de las tablas como la

importación inicial de los espacios culturales y sus índices geoespaciales en el primer arranque.

16. Conclusiones.

El desarrollo de BairesGuessr nos permitió cumplir con el objetivo central de la materia: integrar de manera práctica los principales conceptos de bases de datos relacionales y no relacionales dentro de una aplicación completa y funcional. Partiendo de un conjunto real de datos abiertos del Gobierno de la Ciudad de Buenos Aires, atravesamos todo el ciclo de vida del tratamiento de la información, desde el análisis exploratorio, la limpieza y el enriquecimiento externo mediante la API de Mapillary, hasta el diseño conceptual, lógico y físico de un sistema respaldado por dos paradigmas de persistencia distintos.

Cada etapa propuesta en la consigna se vio reflejada directamente en la arquitectura final del proyecto. Por un lado, el proceso de normalización hasta Tercera Forma Normal, el Diagrama Entidad-Relación y su implementación en PostgreSQL nos permitieron asegurar la integridad referencial, las validaciones de dominio y la consistencia transaccional en la gestión de usuarios, partidas y rondas. Por otro lado, la adopción de MongoDB materializó las ventajas del modelo documental y del estándar GeoJSON con índices geoespaciales 2dsphere, optimizando las consultas sobre el catálogo de espacios culturales sin necesidad de uniones complejas. Asimismo, la construcción del backend en Kotlin con Spring Boot concretó el desacoplamiento entre la interfaz de usuario en React y la capa de datos, coordinando la interacción entre ambos motores y resolviendo la lógica del juego de forma centralizada.

Como principal aprendizaje, comprobamos en la práctica que ningún motor reemplaza universalmente al otro, sino que el verdadero potencial surge de aprovechar las fortalezas de cada tecnología según la naturaleza de los datos, una lección que quedó especialmente marcada tras replantear la división de responsabilidades luego del checkpoint. De igual manera, la contenerización mediante Docker Compose evidenció la importancia de automatizar y estandarizar los entornos para lograr despliegues reproducibles en distintos sistemas operativos. En definitiva, llevar los fundamentos

teóricos de la materia a un producto interactivo y entretenido consolidó nuestra capacidad para modelar datos con criterio técnico y construir soluciones de software robustas y escalables.